Simulation of Communication Systems

ENG3014 Communication Systems 3
Kartike Sharan (3008620s)

This report covers the four simulation tasks, coded in Python 3.9 with NumPy and Matplotlib in a Jupyter notebook. For each task the key code is shown, followed by what it does, its plots and why they look the way they do.

Task 1: Creating Arrays and Plotting Data

Aim: sample $f(t) = a\sin(2\pi\nu t)$ into an array and plot it for different sample frequencies and numbers of samples.

```
def sine_wave(a, hz, sf, N):
    t = np.arange(N) / sf
    f = a * np.sin(2 * np.pi * hz * t)
    return t, f
```

What the code does. `np.arange(N)` gives the sample numbers $n = 0, \ldots, N-1$, and dividing by the sample frequency `sf` gives the sample times $t_n = n/f_s$. NumPy evaluates the sine at every sample time at once, so `f` holds the N sampled values. With $a = 1$ and $\nu = 5$ Hz, four (f_s, N) pairs were plotted against a finely sampled grey reference wave (Figure 1).

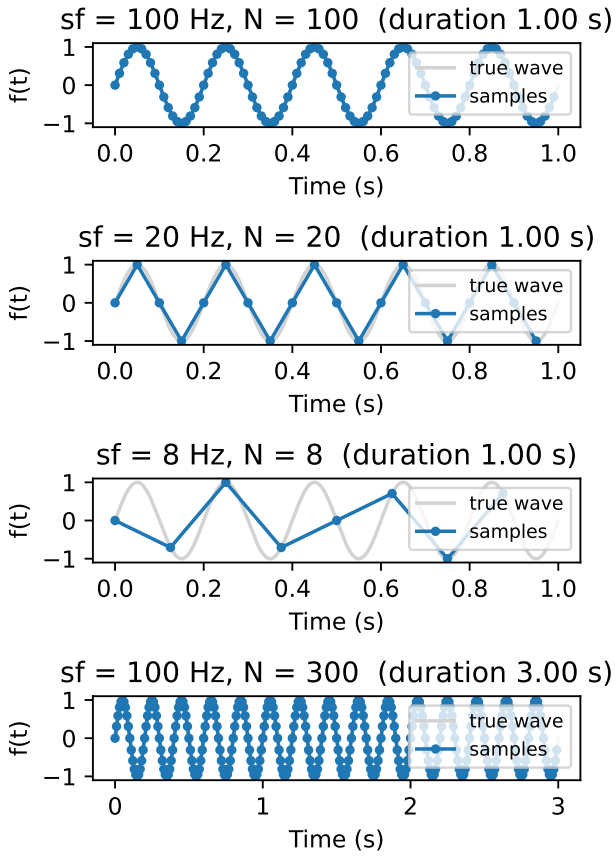


Figure 1: The 5 Hz wave for cases (a) to (d), top to bottom. Grey: reference wave. Blue: samples.

Why the plots look like this.

(a) $f_s = 100$ Hz, $N = 100$: 20 samples per cycle, so the samples follow the wave closely.

(b) $f_s = 20$ Hz, $N = 20$: only 4 samples per cycle, on the peaks and zero crossings, so the joining lines look triangular. The frequency is still right because $f_s > 2\nu$.

(c) $f_s = 8$ Hz, $N = 8$: below $2\nu = 10$ Hz, breaking the Nyquist limit ($f_N = f_s/2 = 4$ Hz). The samples match a 3 Hz wave ($|\nu - f_s| = 3$ Hz) even though every dot lies on the 5 Hz curve: this is aliasing.

(d) $f_s = 100$ Hz, $N = 300$: just a longer record, $T = N/f_s = 3$ s. More samples at the same rate mean more time, not more detail.

Task 2: Modulation of a Carrier Wave with Pure Tones

Aim: write a function for the AM signal $y(t) = [\mathrm{DC} + m\sin(2\pi\nu_m t)]\, A\sin(2\pi\nu_c t)$.

```
def am_signal(A, DC, m, nu_m, nu_c, fs, N):
    t = np.arange(N) / fs
    message = DC + m * np.sin(2 * np.pi * nu_m * t)
    carrier = A * np.sin(2 * np.pi * nu_c * t)
    y = message * carrier
    return t, y
```

What the code does. The sample times are made as in Task 1. `message` is the bracket (message plus DC offset) and `carrier` is the carrier, so multiplying them element by element is the modulation. The plots use $A = \mathrm{DC} = 1$, $\nu_m = 5$ Hz, $\nu_c = 50$ Hz, $f_s = 1$ kHz and $N = 1000$, with $m = 0.5$ and $m = 1.5$.

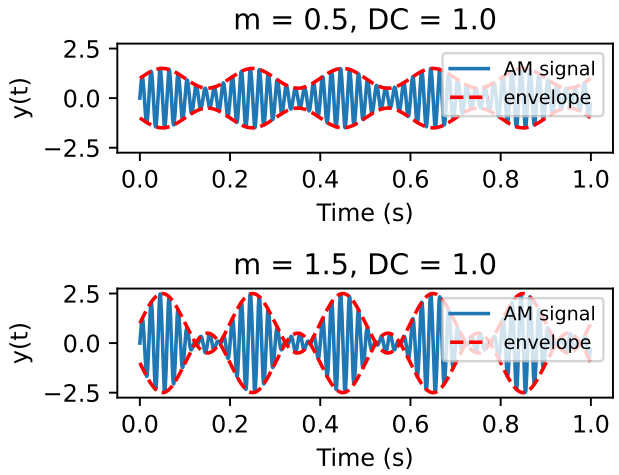


Figure 2: AM signal with $m = 0.5$ (top) and $m = 1.5$ (bottom). Red dashed: the envelope $\pm A[\mathrm{DC} + m\sin(2\pi\nu_m t)]$.

Why the plots look like this. Both plots share one y-axis (`sharey=True`), so their sizes compare directly. The fast oscillation is the 50 Hz carrier, and its peaks rise and fall five times a second with the message. With $m = 0.5$ the bracket stays between 0.5 and 1.5, so the envelope is an exact copy of the message. With $m = 1.5 > \mathrm{DC}$ the bracket goes negative for part of each cycle and flips the carrier there, so the outline no longer follows the message. The DC offset keeps the envelope positive, so m must not exceed DC.

Task 3: Discrete Fourier Transform (DFT)

Aim: compute the DFT of the AM signal with a hand-written function, scale the axes to real units, explore f_s, N, ν_c and ν_m, and compare the result with NumPy's FFT.

```
def my_dft(f):
    N = len(f)
    n = np.arange(N)
    F = np.zeros(N, dtype=complex)
    for k in range(N):
        F[k] = np.sum(f * np.exp(-1j * 2 * np.pi * k * n / N)) / N
    return F

def freq_axis(N, fs):
    return np.arange(N) * fs / N
```

What the code does. `my_dft` is the lab formula, $F_k = \frac{1}{N}\sum_{n=0}^{N-1} f_n\, e^{-i2\pi kn/N}$: for each bin k it multiplies every sample by the complex exponential, adds them up and divides by N. `freq_axis` converts bin k to hertz with $\nu_k = k f_s/N$, and only the first half of the

array, up to f_{N}, is plotted. The $1/N$ puts the vertical axis in real units too: a tone of amplitude B gives a peak of height $B/2$.

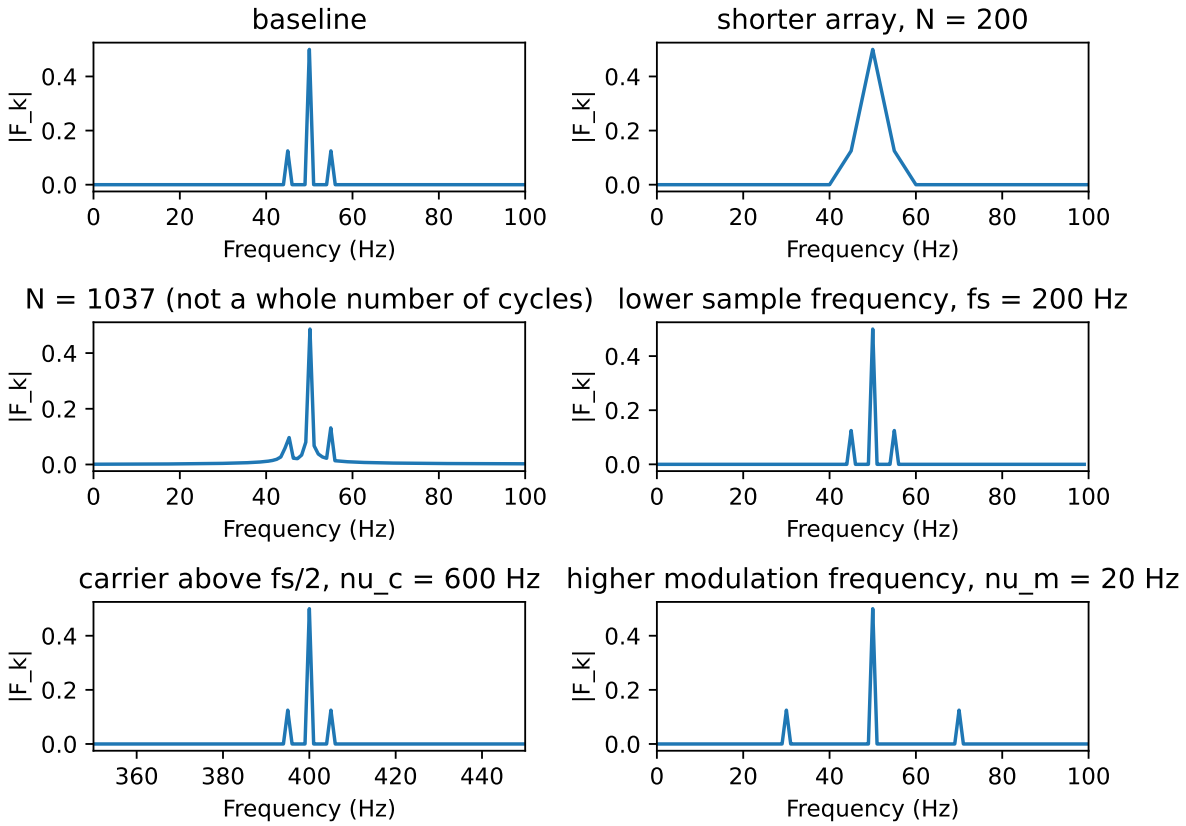


Figure 3: DFT for six cases, (a) to (f) left to right and top to bottom, each changing one thing from the baseline. Each plot is zoomed to where its peaks are.

Why the plots look like this. Expanding the AM signal gives three tones: the carrier at ν_c (amplitude $A \cdot \mathrm{DC}$) and two sidebands at $\nu_c \pm \nu_m$ (amplitude $Am/2$).

(a) Baseline: peaks of 0.125, 0.5 and 0.125 at 45, 50 and 55 Hz, exactly as predicted. Each tone's peak is half its amplitude.

(b) $N = 200$: coarser resolution, $\Delta f = f_{\mathrm{s}}/N = 5$ Hz, so the peaks sit in neighbouring bins and merge into one broad peak.

(c) $N = 1037$: the tones no longer complete a whole number of cycles, so they fall between bins and leak. The peaks drop and spread into a low skirt.

(d) $f_{\mathrm{s}} = 200$ Hz: f_{N} falls to 100 Hz, but 55 Hz is still below it, so nothing changes.

(e) $\nu_c = 600$ Hz: above $f_{\mathrm{N}} = 500$ Hz, so the cluster aliases to $1000 - 600 = 400$ Hz.

(f) $\nu_m = 20$ Hz: the sidebands move to 30 and 70 Hz, so the bandwidth grows to $2\nu_m = 40$ Hz.

FFT comparison. This code times `my_dft` and NumPy's FFT on the baseline signal (`time.perf_counter()` reads the clock before and after each one), then compares the two results. NumPy's FFT has no $1/N$, so it is divided by `N` to match.

```
start = time.perf_counter()
F_mine = my_dft(y)
t_mine = time.perf_counter() - start

start = time.perf_counter()
F_fft = np.fft.fft(y) / N
t_fft = time.perf_counter() - start

print("Same result?", np.allclose(F_mine, F_fft))
print("Largest difference:", np.max(np.abs(F_mine - F_fft)))
print(f"my_dft: {t_mine*1e3:.2f} ms    np.fft.fft: {t_fft*1e3:.3f} ms")
```

Printed output

```
Same result? True
Largest difference: 4.2816584692812026e-14
my_dft: 18.36 ms    np.fft.fft: 1.969 ms
```

`np.allclose` returned `True` and the largest difference is 4.3×10^{-14}, which is just rounding error, so `my_dft` gives the same spectrum as the library FFT. The FFT took 1.97 ms against 18.36 ms for `my_dft`, about 9 times faster. It needs about $N \log_2 N \approx 10^4$ operations for $N = 1000$, against $N^2 = 10^6$ for the direct sum.

Task 4: Signal Filtering

Aim: filter the Task 2 signal with a square band-pass filter in k-space, return to the time domain with the inverse FFT, and try several bandwidths.

```
def bandpass(y, fs, f_low, f_high):
    N = len(y)
    Y = np.fft.fft(y)
    freqs = np.fft.fftfreq(N, d=1/fs)
    H = ((np.abs(freqs) >= f_low) & (np.abs(freqs) <= f_high)).astype(float)
    y_filt = np.fft.ifft(Y * H).real
    return y_filt, freqs, Y, H
```

What the code does. `np.fft.fft` moves the signal to the frequency domain, and `fftfreq` gives each bin's signed frequency. `H` is 1 where $f_{\text{low}} \leq |f| \leq f_{\text{high}}$ and 0 elsewhere: the square filter, applied to positive and negative frequencies alike so the output stays real. `Y * H` applies the filter, which is the same as convolving the signal with the filter in time, and `np.fft.ifft` returns the filtered signal. Three pass bands were tried (Figure 4).

Why the plots look like this. **48–52 Hz** keeps only the carrier, so the output is a constant 50 Hz sine and the message is lost. **40–60 Hz** keeps all three tones, so the output is identical to the original. **48–60 Hz** removes the lower sideband, so the envelope still varies at 5 Hz but only between 0.75 and 1.25, half the original swing. The message is carried in the sidebands, so the pass band must be at least $2\nu_m$ wide to keep all of it.

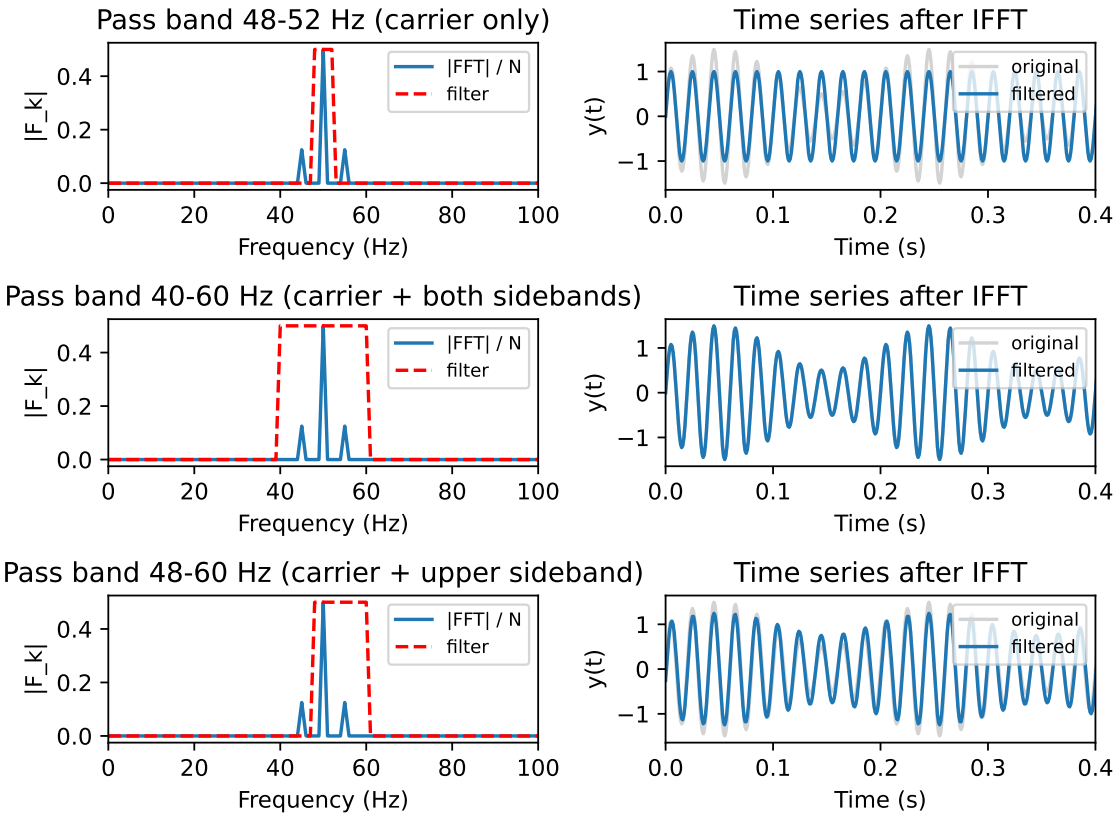


Figure 4: Spectrum with the filter (left) and the time series after the IFFT (right) for pass bands of 48–52, 40–60 and 48–60 Hz, top to bottom.

Conclusion

All four tasks were completed, and each plot matches the theory. The sample frequency must be more than twice the highest frequency to avoid aliasing. AM puts the message into sidebands at $\nu_c \pm \nu_m$. The DFT's resolution is set by f_s/N, and its axes can be scaled to hertz and signal units. Multiplying by a square window in k-space filters the signal.